
Teaching Verification-Centered Software Engineering with Coding Agents

Tiejian Luo^{2,*} Chenxi Luo¹ Moon-Kuen Mak² Wei Xiang¹

¹La Trobe University ²University of Chinese Academy of Sciences

*Corresponding author: tjluo@ucas.ac.cn

[tjluo-ucas.github.io/nips-edu-2026](https://github.com/tjluo-ucas.github.io/nips-edu-2026)

Abstract

As coding agents undertake longer software tasks, education must address how engineers can trust the resulting changes. We teach *verification-centered software engineering*: validate intent, constrain the agent interface, challenge a candidate with independent executable evidence, revise from counterexamples, scope the supported claim, and make a separate architecture/security approval decision. EvalPlus, SWE-agent, CLEVER, and ReLoc motivate four failure surfaces. Our learner—not the agent—is the primary object of improvement.

1 The Concept

Unlike self-improving-agent courses, which optimize model, search, reward, or planner performance, this module holds the agent fixed or bounded and improves the engineer’s evidence and judgment. Its lifecycle is:

$$I \xrightarrow{\text{validate}} S \rightarrow (\text{Agent} + \text{ACI}, X) \rightarrow C \xrightarrow{V} \{E, F\},$$

$$\{E, F\} \rightarrow R \rightarrow C' \text{ (repeat)} \rightarrow K \xrightarrow{J} \{\text{approve, reject}\}.$$

Here intent I becomes specification S ; agent and interface act in environment X ; verifier V challenges candidate C ; and evidence/counterexample informs revision R or property-scoped claim K . Learners distinguish **correctness evidence** (what property is supported), **revision guidance** (what to try next), and **approval judgment** J (whether the change should enter the lifecycle); model explanation is separately classified as opinion, not independent evidence. Passing V is neither universal correctness nor approval.

2 Leveling and Prerequisite Knowledge

The resource targets **advanced undergraduate and graduate** AI/SE learners with basic Python, Git/diff, and unit-testing knowledge; no agent framework, LLM API, reinforcement learning, or formal methods are required. The 150-minute workshop spans specification through authorization transfer. A provenance-marked prepared candidate makes it reproducible without paid model access.

3 Learning Objectives and Outcomes

By the end, learners will **understand** signal categories; **apply** the lifecycle under a bounded agent–computer interface; **analyze** a weak verifier with boundaries and mutations; **evaluate** a scoped claim separately from architecture/security approval; and **create** an authorization-domain transfer solution. Assessment measures epistemic calibration and engineering judgment through anchored questions, a withheld suite, evidence/decision artifacts, and non-isomorphic transfer—not agent-building proficiency, prompts, tokens, or generated-code volume.

4 Linked Papers

The papers establish four complementary premises and map directly to assessed activities.

EvalPlus: weak evidence. HumanEval+ expands tests $80\times$; across 26 models, stricter tests reduce $\text{pass}@k$ by up to 19.3–28.9% and can reverse rankings (Liu et al., 2023). Learners kill a mutation with boundary evidence, then replace “correct” with a suite-scoped claim.

SWE-agent: interface design. Its LM-oriented ACI provides search/edit actions, guardrails, and concise feedback; its Lite result is a 64% relative gain over a shell-only agent with the same model (Yang et al., 2024). Learners hold the task fixed, compare interface contracts, and specify actions, observations, guardrails, and escalation.

CLEVER: specification fidelity. For 161 curated Lean tasks, CLEVER checks generated-specification equivalence and implementation correctness; evaluated pipelines solve at most one task end-to-end (Thakur et al., 2025). Our accessible analogue separates stakeholder validation of S from execution of V : a checker can certify the wrong property.

ReLoc: revision guidance. ReLoc decomposes improvement into drafting, neighborhood generation, evaluation, and incumbent update; revision distance guides search when public-test rewards are sparse (Lyu et al., 2025). Learners classify such a score as guidance, record each candidate/evidence state, and never relabel search preference as correctness evidence.

Our pedagogical synthesis adds a fifth layer: **claim calibration and engineering approval**. More search may improve C ; it does not define the evidence required to accept the change.

5 Teaching Materials Summary

Following a failure-surface $\rightarrow$ research evidence $\rightarrow$ minimal demonstration $\rightarrow$ learner action $\rightarrow$ executable artifact $\rightarrow$ transfer sequence, the package provides slides/PPTX, notebook, provenance-marked patch, weak/withheld suites, test-equivalent architectural alternatives, authorization transfer, contracts, ledgers, rubrics, and a pilot protocol. Staged bundles separate worked answers; one build tests and packages the resource. No human learning outcome is claimed.

References

- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. NeurIPS 2023.
- John Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024.
- Amitayush Thakur et al. *CLEVER: A Curated Benchmark for Formally Verified Code Generation*. NeurIPS 2025, Datasets and Benchmarks Track.
- Zhiyi Lyu et al. *Let’s Revise Step-by-Step: A Unified Local Search Framework for Code Generation with LLMs*. NeurIPS 2025.